

Campus Trading Application: Sharding Implementation Report

CS 432: Databases — IIT Gandhinagar Group 8 – Optimiser

Bhavik Patel
22110047
IIT Gandhinagar

Hitesh Kumar
22110098
IIT Gandhinagar

Junil Patel
22110184
IIT Gandhinagar

Pranav Patil
22110199
IIT Gandhinagar

Sibtain Raza
22110148
IIT Gandhinagar

Abstract

This report details the implementation of Assignment 4 (includes Task 1) for the *Campus Trading Application* for course DBMS, extending the existing MySQL / Go / React stack with horizontal scaling through sharding. The implementation uses a modulo-based shard key:

$$\text{shard_id} = \text{record_id} \bmod 3$$

Partitioned marketplace tables are distributed across three database shards, while extttMember and the low-usage control-plane tables remain centralised on Shard 1. The Category table is replicated to every shard.

The report covers shard-key selection, partitioning strategy, query routing, migration, isolation, and the trade-offs of the chosen design. The backend routes direct lookups to the owning shard, while browse-style queries fan out across all shards and merge the results in application code.

The final design achieved balanced data placement, clear shard isolation, and successful migration without duplicated rows.

GitHub Repository

Video Demonstration Link

Keywords

Database Sharding, Horizontal Scaling, Query Routing, MySQL, Distributed Databases

1 Shard Key Chosen and Justification

The implementation uses a simple modulo-based routing rule:

$$\text{shard_id} = \text{record_id} \bmod 3$$

This shard key satisfies the three required criteria:

- **High Cardinality:** primary keys are unique and naturally spread across a large value space.
- **Query-Aligned:** the application usually fetches listings, offers, transactions, threads, and marketplace data by ID. User records in Member are centralised and therefore do not participate in modulo routing.
- **Stable:** once assigned, primary keys never change.

Modulo routing spreads consecutive IDs evenly across the three shards. The partitioned tables therefore stay close to balanced.

2 Partitioning Strategy Used and Why

The project uses hash-style modulo partitioning rather than range-based or directory-based partitioning.

- It is deterministic and easy to explain.
- It keeps shard selection $O(1)$ for a known ID.
- It avoids skew caused by clustering low or high IDs onto one shard.
- It matches the assignment requirement to simulate sharding using multiple databases.

2.1 Shard Endpoints

The deployment uses three shard endpoints:

- Shard 1: 10.0.116.184:3307
- Shard 2: 10.0.116.184:3308
- Shard 3: 10.0.116.184:3309

2.2 Table Placement

Listing 1: Table routing map (backend/sharding/router.go)

```
1 var tableRoutes = map[string]TableRoute{
2     "Administrator": {TableName: "Administrator", KeyColumn: "
3         AdminID", Placement: PlacementCentral},
4     "Category": {TableName: "Category", KeyColumn: "
5         CategoryID", Placement: PlacementReplicate},
6     "audit_log": {TableName: "audit_log", KeyColumn: "log_id
7         ", Placement: PlacementCentral},
8     "sys_role": {TableName: "sys_role", KeyColumn: "role_id
9         ", Placement: PlacementCentral},
10    "sys_session": {TableName: "sys_session", KeyColumn: "
11        session_id", Placement: PlacementCentral},
12    "sys_user": {TableName: "sys_user", KeyColumn: "user_id
13        ", Placement: PlacementCentral},
14    "sys_user_role": {TableName: "sys_user_role", KeyColumn: "
15        user_id", Placement: PlacementCentral},
16    "Member": {TableName: "Member", KeyColumn: "MemberID"
17        , Placement: PlacementCentral},
18    "WishRequest": {TableName: "WishRequest", KeyColumn: "
19        WishRequestID", Placement: PlacementPartition},
20    "Listing": {TableName: "Listing", KeyColumn: "
21        ListingID", Placement: PlacementPartition},
22    "ListingImage": {TableName: "ListingImage", KeyColumn: "
23        ImageID", Placement: PlacementPartition},
24    "Offer": {TableName: "Offer", KeyColumn: "OfferID",
25        Placement: PlacementPartition},
26    "Transaction": {TableName: "Transaction", KeyColumn: "
27        TransactionID", Placement: PlacementPartition},
28    "MessageThread": {TableName: "MessageThread", KeyColumn: "
29        ThreadID", Placement: PlacementPartition},
30    "Message": {TableName: "Message", KeyColumn: "
31        MessageID", Placement: PlacementPartition},
32    "Notification": {TableName: "Notification", KeyColumn: "
33        NotificationID", Placement: PlacementPartition},
34    "Watchlist": {TableName: "Watchlist", KeyColumn: "
35        WatchlistID", Placement: PlacementPartition},
36    "Report": {TableName: "Report", KeyColumn: "ReportID"
37        , Placement: PlacementPartition},
38    "Rating": {TableName: "Rating", KeyColumn: "RatingID"
39        , Placement: PlacementPartition},
40 }
```

Category is replicated to every shard because it is small and frequently used in joins.

The low-usage control-plane tables remain centralised on Shard 1 only. These include Administrator, Member, sys_user, sys_role, sys_user_role, sys_session, and audit_log.

Keeping Member on Shard 1 simplifies authentication, seller lookup, and cross-listing ownership checks because all user records remain in one place.

3 How Query Routing Is Implemented

3.1 Connection Architecture

The backend opens one connection pool for the source database and one connection pool for all shard targets.

Listing 2: Shard connection setup (backend/db/db.go)

```
1 sharding.Configure(baseDatabase, sharding.DefaultShardCount,
2   shardTargets)
3 Shards = openShardConnections(dsn, baseDatabase,
4   shardTargets,
5   shardUser,
6   shardPassword)
```

3.2 Shard Selection Logic

Listing 3: Modulo-based shard selection

```
1 func (r *Router) ShardIDFor(recordID int) int {
2   if r == nil || r.shardCount <= 0 {
3     return 0
4   }
5   shardID := recordID % r.shardCount
6   if shardID < 0 {
7     shardID += r.shardCount
8   }
9   return shardID
10 }
```

For direct lookups and writes, the backend routes marketplace records to the owning shard. Queries involving Member are always sent directly to Shard 1 because that table is centralised.

For browse-style queries, the backend queries all shards and merges the results in application code.

Listing 4: Browse query fan-out across shards

```
1 func fetchListingRowsAcrossShards(ctx context.Context,
2   baseWhere string, args []interface{}) ([]listingRow, error) {
3   {
4     query := `SELECT ListingID, SellerID, CategoryID, Title,
5       Description, AskingPrice, IsNegotiable,
6       ` + "`Condition`" + `, Status,
7       CreatedDate, LastModifiedDate,
8       WishRequestID
9     FROM Listing l` + baseWhere
10
11   var rows []listingRow
12   for _, shardDB := range appdb.AllShardConnections() {
13     shardRows, err := shardDB.QueryContext(ctx, query, args
14       ...)
15     // merge rows ...
16   }
17   return rows, nil
18 }
```

Query type	Routing behaviour	Example
Single lookup	Direct to owning shard	GET /listings/{id}
Single insert	Allocate ListingID, then route	POST /listings
Related write	Route using parent listing shard	POST /listings/{id}/offers
Browse range	/ Fan-out and merge	GET /listings

4 SQL Shard Tables Created and Data Migration

Migration is handled by scripts/migrate_shards.py.

- (1) Create the shard schema on each target shard.
- (2) Read rows from the source database.
- (3) Route partitioned rows using modulo routing.
- (4) Copy control-plane tables only to Shard 1.
- (5) Replicate Category to all shards.
- (6) Commit writes and verify row counts.

Listing 5: Migration routing logic

```
1 def copy_partitioned_table(source_cursor, shard_connections,
2   shard_configs, router,
3   base_database, table):
4   columns, rows = fetch_primary_rows(source_cursor,
5     base_database, table)
6   for row in rows:
7     routing_value = route_value_for_table(table, row,
8       columns)
9     shard_id = router.shard_id_for(routing_value)
10    # insert into shard_connections[shard_id]
11
12 def copy_central_table(source_cursor, shard_connections,
13   shard_configs, base_database, table):
14   columns, rows = fetch_primary_rows(source_cursor,
15     base_database, table)
16   shard_connection = shard_connections[0]
17   # insert rows only into shard 1
18
19 def copy_replicated_table(source_cursor, shard_connections,
20   shard_configs, base_database, table):
21   columns, rows = fetch_primary_rows(source_cursor,
22     base_database, table)
23   for shard_id, shard_connection in enumerate(
24     shard_connections):
25     # insert rows into every shard
```

4.1 Centralised and Replicated Tables

The final shard layout uses three placement modes:

- **Partitioned:** marketplace tables except Member are distributed using modulo routing.
- **Replicated:** Category is copied to every shard.
- **Centralised:** Member and the control-plane tables remain only on Shard 1.

5 Verification

Verification is performed by scripts/verify_shards.py.

- Partitioned-table totals across all shards match the source database.
- No partitioned primary key appears on more than one shard.
- Replicated tables appear on every shard.
- Centralised tables appear only on Shard 1.

6 Scalability and Trade-Offs Analysis

6.1 Horizontal vs. Vertical Scaling

Vertical scaling upgrades one database server with more CPU, memory, and storage. It is simple but leaves one server as the bottleneck.

Horizontal scaling with shards spreads data and traffic across multiple databases, allowing the system to grow by adding capacity.

6.2 Consistency

Each shard remains transactional on its own. However, the design does not provide global transactions across shards.

Cross-shard reads may therefore be temporarily inconsistent, and replicated data may briefly differ if one shard is updated before another.

6.3 Availability

If one shard fails, only the data mapped to that shard becomes unavailable. The other shards continue serving their own data.

6.4 Partition Tolerance

The design tolerates partial shard failure because requests are routed only to the owning shard.

7 Observations and Limitations

7.1 Observations

The modulo shard key produced an even spread for the partitioned marketplace tables. The verified counts stayed close across all three shards, while Member and the other centralised control-plane tables remained only on Shard 1.

Once the shard endpoints were configured, migration was straightforward. Partitioned marketplace tables were routed to their owning shard, while Member and the other control tables were copied only to Shard 1. Category was replicated to every shard.

The verification step also confirmed that Shard 2 and Shard 3 do not contain the centralised control tables.

Single-record lookups and writes were easy to route directly, but browse-style queries still required fan-out and merge logic.

7.2 Limitations

- **No cross-shard transactions:** writes touching multiple shards are not globally atomic.
- **Fan-out cost:** browse queries and reports still require reading from every shard.
- **No automatic failure recovery:** there is no built-in retry or failover logic.
- **No live rebalancing:** changing the shard count requires a full re-migration.
- **Centralised-table dependency:** if Shard 1 fails, authentication, member lookups, and administrative features become unavailable because Member and the control-plane tables are stored only there. This can be fixed but will cost repetition of data on all the shards.

8 Author Contributions

All five members contributed equally to the sharding design, implementation, migration, testing, and report writing.

- **Bhavik Patel** — Shard-key analysis, data distribution verification, and skew analysis.
- **Hitesh Kumar** — Go router implementation, shard endpoint configuration, and query routing.
- **Jinil Patel** — Frontend adaptation and shard-aware API integration.
- **Pranav Patil** — Shard schema creation, migration scripts, and central-table handling.
- **Sibtain Raza** — Verification scripts, scalability analysis, and final report editing.

Acknowledgments

We acknowledge Professor Yogesh Meena and the teaching assistants for their guidance throughout the assignment.